

Bridging the Sim-to-Real Gap in Quadrupedal Visual Navigation via 3D Gaussian Splatting

Minseok Lee¹, Seongyu Han¹, Heejae Moon¹, and Sung Soo Hwang^{1*}

¹School of Artificial Intelligence, Computer and Electrical Engineering,
Handong Global University, Pohang 37554, Republic of Korea
({glen, tjsrb439, 22000249}@handong.ac.kr, sshwang@handong.edu) * Corresponding author

Abstract: Deploying visual navigation on physical quadrupedal robots is challenging due to the visual sim-to-real gap and locomotion-induced camera jitter. This jitter degrades feature tracking and depth consistency. This paper presents a pipeline that bridges these perceptual and dynamic gaps to enable virtual parameter optimization. In the simulation stage, we reconstruct a 3D Gaussian Splatting (3DGS) digital twin of a campus corridor and use a reinforcement learning (RL) locomotion policy to generate camera bobbing noise. Under these simulated visual and dynamic disturbances, the parameters of the RTAB-Map visual SLAM and Nav2 stacks are optimized. For deployment, the RL policy is bypassed, and velocity commands are routed directly to the physical robot’s built-in Sport API via a ROS 2 bridge. Real-world experiments demonstrate that parameters optimized under simulated jitter transfer directly to the physical platform, supporting visual mapping and qualitative obstacle-aware replanning without manual recalibration.

Keywords: Quadrupedal Locomotion, Visual SLAM, Sim-to-Real Pipeline, 3D Gaussian Splatting, Ego-motion Jitter, Virtual Sandbox.

1. INTRODUCTION

Autonomous quadrupedal robots offer superior mobility over wheeled systems on challenging terrains. Achieving indoor autonomy requires integrating legged locomotion with simultaneous localization and mapping (SLAM). While indoor navigation has conventionally relied on LiDAR sensors, they are heavy and power-intensive. Visual navigation using a single RGB-D camera is a lightweight, cost-effective alternative.

However, legged visual navigation introduces two primary sim-to-real challenges. First is the *perceptual gap*: standard simulators lack photorealistic textures and depth fidelity, causing visual SLAM to fail in the real world due to feature matching errors. Second is the *dynamic gap*: legged locomotion generates camera bobbing that causes motion blur and false obstacle detection (phantom obstacles). Furthermore, tuning parameters directly on physical hardware is limited by battery capacity and collision risks.

To address these challenges, we present a pipeline that decouples simulation-based parameter verification from physical deployment. Using a single front-facing RGB-D camera (Intel RealSense D435i) and offline 3DGS reconstruction from DSLR images, we align virtual and physical camera properties to reduce sensor discrepancy. In simulation, we reconstruct a digital twin of an indoor corridor. Since the robot’s low-level controller is proprietary, we train an RL locomotion policy in Isaac Lab to emulate physical gait oscillations, generating representative camera noise. In this virtual environment, we optimize the RTAB-Map and Nav2 parameters. During physical deployment, the computationally intensive RL policy is bypassed, and velocity commands are routed directly to the manufacturer’s Sport API via a ROS 2 bridge, conserving edge computing resources for SLAM and path planning.

In summary, this work establishes a photorealistic val-

idation environment by integrating DSLR-based 3DGS and *fVDB* reconstruction into a physics simulator while aligning the virtual camera with the physical sensor. It further uses an RL locomotion policy to emulate legged walking dynamics and verifies on a physical Unitree Go2 that parameters optimized under simulated disturbances can transfer without recalibration.

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 describes the system architecture, locomotion learning, and environment construction. Section 4 details the navigation configuration, simulated validation, and real-world deployment results. Finally, Section 5 concludes this paper.

2. RELATED WORK

Reinforcement learning (RL) has advanced legged control [1]. Prior studies have transferred quadrupedal policies through actuator modeling, domain randomization, and massively parallel training [6], [7]. However, most legged RL research focuses on gait stability, leaving visual navigation integration and sensor noise characteristics unaddressed.

Mobile robot navigation has traditionally relied on LiDAR sensors, which are heavy and power-intensive for payload-constrained quadruped robots. Visual SLAM (VSLAM) using RGB-D cameras offers a lightweight alternative, with ORB-SLAM2 [8] and RTAB-Map [2] providing practical foundations for camera-based mapping. For path planning and obstacle avoidance, the ROS 2 Navigation (Nav2) framework [3] is widely used.

Photorealistic reconstruction methods provide another route to reducing the perceptual sim-to-real gap. Structure-from-Motion pipelines such as COLMAP [9] estimate camera poses, while 3DGS [4] enables real-time photorealistic rendering from reconstructed scenes. Al-

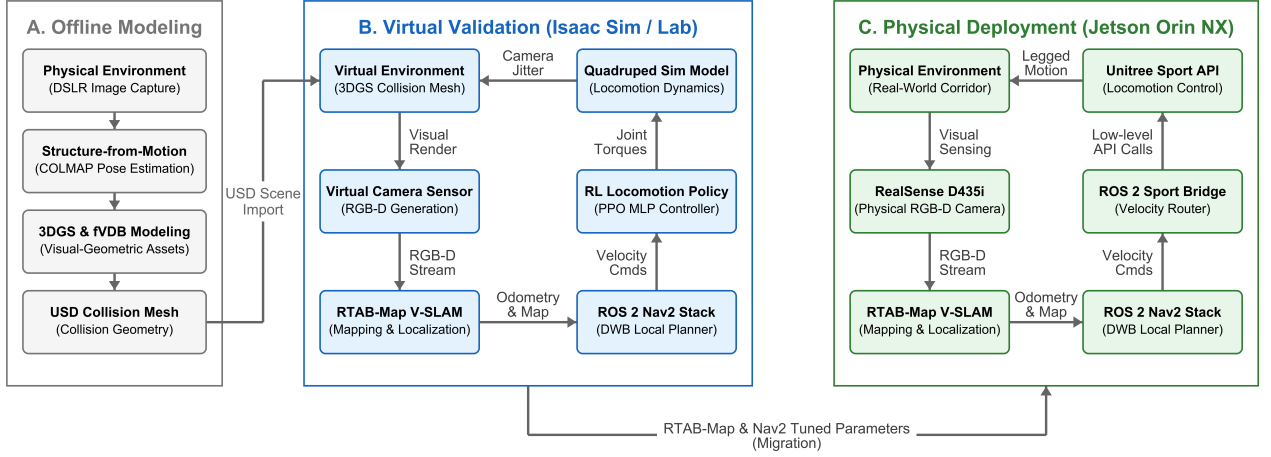


Fig. 1. Overall architecture of the proposed Sim-to-Real quadrupedal navigation framework. (a) In the virtual validation stage, we reconstruct a 3DGS environment and train an RL locomotion policy to emulate camera bobbing noise. (b) In the physical deployment stage, the RL policy is bypassed, and commands are sent to the Unitree Sport API via a ROS 2 bridge to reduce computational load on the Jetson Orin NX.

though frameworks like VR-Robo [5] use 3DGS to build digital twins, they often rely on heterogeneous sensors, introducing device-level discrepancies in lens distortion and field-of-view (FOV). In contrast, our pipeline configures the virtual camera to match the physical D435i sensor, minimizing visual discrepancies.

3. SYSTEM AND LOCOMOTION CONTROL

3.1 System Architecture

Our framework integrates RTAB-Map, Nav2, and a dual-stage locomotion control architecture. In simulation, a DRL policy—trained via PPO in Isaac Lab (v2.3.2) on Isaac Sim (v5.1.0)—generates camera bobbing noise. High-level velocity commands ($\mathbf{v}_{\text{cmd}} = [\mathbf{v}_x, \mathbf{v}_y, \omega_z]^T$) from the Nav2 local planner are mapped directly to the policy’s action space. During real-world deployment, the DRL policy is bypassed, and velocity commands are routed directly to the Unitree Sport API via a ROS 2 bridge. This architecture reduces the computational load on the Jetson Orin NX. The system architecture is shown in Fig. 1.

3.2 Locomotion Learning in Isaac Sim

To simulate camera bobbing noise, we train an RL policy in NVIDIA Isaac Lab (v2.3.2) on the Isaac Sim (v5.1.0) physics engine. The policy functions to generate walking-induced oscillations. We construct a training environment with mixed terrains (uneven ground, ramps, step obstacles) using the Procedural Terrain Generator and scale terrain difficulty via curriculum learning. To enhance policy robustness, we apply the following domain randomizations:

- Torso mass randomization: $\Delta m \in [-1.0, +3.0]$ kg.
- Ground friction coefficient: $\mu \in [0.6, 0.8]$.
- External disturbances: impulse force perturbations applied to the robot’s center of mass (CoM).

The physics step is 200 Hz, and the policy executes at

50 Hz. Training uses the PPO implementation in the PyTorch-based *skrl* library. Both actor and critic networks are MLPs with hidden layers of [512, 256, 128] and ELU activation functions. The learning rate is 1.0×10^{-3} , regulated by a KL-divergence scheduler. The discount factor is $\gamma = 0.99$, and the GAE parameter is $\lambda = 0.95$.

The reward function tracks velocity references while maintaining legged stability. The total reward R is:

$$R = w_{lin}R_{lin} + w_{ang}R_{ang} + w_{air}R_{air} - \sum_i w_{p,i}P_i \quad (1)$$

where:

- $R_{lin} = \exp(-\|\mathbf{v}_{xy}^* - \mathbf{v}_{xy}\|_2^2 / \sigma_{lin}^2)$ is the linear velocity tracking reward ($\mathbf{v}_{xy}^*$ and $\mathbf{v}_{xy}$ are target and actual velocities, $w_{lin} = 1.5$, $\sigma_{lin} = 0.25$).
- $R_{ang} = \exp(-(\omega_z^* - \omega_z)^2 / \sigma_{ang}^2)$ is the angular velocity tracking reward (ω_z^* and ω_z are target and actual yaw rates, $w_{ang} = 0.75$, $\sigma_{ang} = 0.25$).
- R_{air} is the foot clearance air-time reward ($w_{air} = 0.25$).
- P_i represents penalty terms for base vertical velocity, roll/pitch oscillations, joint limits, excessive torque, joint acceleration, and unintended contact ($w_{p,i}$ are penalty weights).

3.3 3DGS Environment Construction & Agent Deployment

We capture corridor images using a DSLR camera to avoid compression artifacts. These images serve as input to COLMAP [9] for Structure-from-Motion (SfM) to resolve camera poses. Then, a 3DGS [4] model is optimized and exported as a USDZ asset for visual rendering, alongside a PLY mesh for coarse geometry. For physical collision modeling, we integrate the scene with NVIDIA’s sparse volumetric framework, *fVDB*. The resulting digital twin, combining visual photorealism and physical collision boundaries, is shown in Fig. 2. We utilize the high-fidelity 3DGS model for camera rendering (Fig. 2, right) and the corresponding fVDB

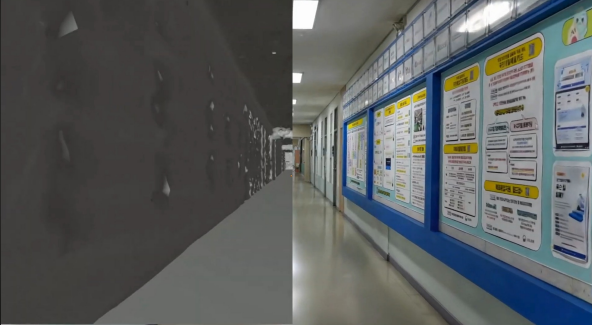


Fig. 2. Digital twin representation of the campus corridor. Left: Sparse volumetric collision mesh generated using fVDB. Right: 3DGS render.

sparse volumetric mesh for collision detection and robot-environment contact simulation (Fig. 2, left).

We import these assets into Isaac Sim, where the collision boundaries are computed using the *fVDB* representation. A ROS 2 bridge publishes virtual RGB-D sensor streams, visualizes the robot state in RViz, and routes velocity commands. The virtual camera’s field-of-view and resolution are configured to match the physical D435i camera to reduce the visual gap. The pre-trained locomotion policy is then deployed on the Go2 model to maintain stable gaits within the simulated corridor.

4. EXPERIMENTAL RESULTS

4.1 Experimental Setup and Evaluation Methodology

The navigation system consists of an Intel RealSense D435i RGB-D sensor, RTAB-Map [2], and the ROS 2 Nav2 stack. The sensor is mounted to the front torso of the Go2, and the virtual camera in simulation replicates its color and depth profiles to ensure consistency. Unlike monocular frameworks (e.g., VR-Robo [5]) that suffer from depth ambiguity, the active depth sensor provides direct geometric measurements for local costmaps.

RTAB-Map uses visual keypoints (e.g., GFTT/ORB) for odometry and loop-closure detection. The depth stream is projected to construct a 3D octomap and a 2D occupancy grid. For low-latency obstacle avoidance, raw depth frames are converted to a pseudo-2D laser scan via the *depthimage_to_laserscan* package. The Nav2 global planner plans paths on the occupancy grid, while the DWB local planner uses the local costmap to compute steering commands $\mathbf{v}_{cmd}$.

To resolve the coordinate transform (TF) tree discrepancy, a custom bridge maps the tree as $map \rightarrow odom \rightarrow world \rightarrow base$, where RTAB-Map publishes the $map \rightarrow odom$ transform and Isaac Sim links odometry to the world frame. This enables standard ROS 2 navigation in simulation. The system is evaluated by analyzing mapping consistency in both simulated and physical environments and qualitatively inspecting obstacle-aware replanning in simulation under locomotion-induced camera oscillations.

4.2 Results and Evaluation

We validated the navigation stack in a simulated 3DGS corridor (80 m long, 2.5 m wide) before physical deployment.

In simulation, the locomotion policy receives proprioceptive and exteroceptive (height-scan) inputs. Ray-casting queries the collision mesh to provide local elevation profiles for foot placement. The closed-loop navigation system in simulation is illustrated in Fig. 3. The local planner (DWB) in the Nav2 stack generates velocity commands based on the synchronized costmap and path planning displayed in RViz (Fig. 3, right). These commands are then sent to the trained RL locomotion policy controlling the virtual robot in Isaac Sim (Fig. 3, left), which simulates the physical walking dynamics and camera motion. Virtual obstacles are inserted in the corridor to qualitatively verify obstacle-aware replanning. As shown in the RViz visualization (Fig. 3, right), the local planner recalculates paths around the inserted obstacles, and the simulated robot follows the replanned trajectory (Fig. 3, left).

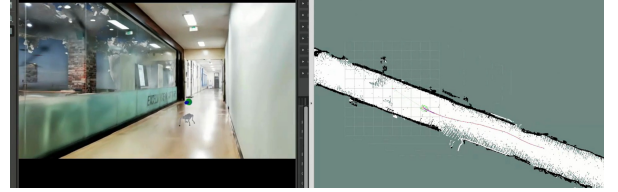


Fig. 3. Autonomous navigation execution inside the reconstructed virtual environment, showing the simulated quadruped robot in Isaac Sim (left) and path planning/costmap synchronization in RViz (right).

The resulting mapping outputs generated during simulated navigation are shown in Fig. 4.

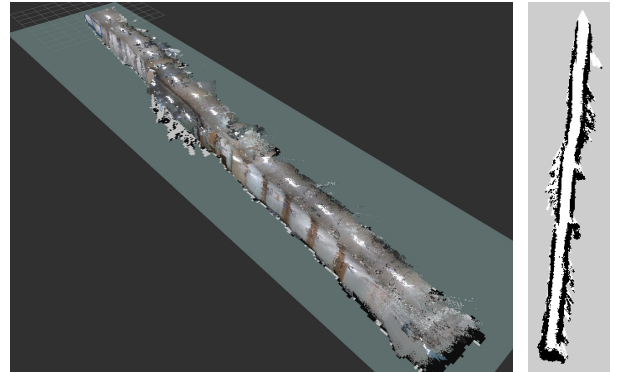


Fig. 4. Visual mapping results in simulation. Left: Reconstructed 3D point cloud map from RTAB-Map. Right: Projected 2D occupancy grid map.

We deployed the proposed framework on a physical Unitree Go2 robot to navigate the campus corridor, as shown in Fig. 5. The robot is equipped with a front-facing Intel RealSense D435i camera and an NVIDIA Jetson Orin NX computer, running the entire navigation and mapping stack locally.

To reduce computational load on the Jetson Orin NX,

the RL policy is bypassed, and commands are sent to the built-in Sport API via a ROS 2 bridge, reserving onboard resources for SLAM and path planning. The camera is connected via USB 3.0.



Fig. 5. Real-world experimental setup. The Unitree Go2 robot is equipped with an Intel RealSense D435i and Jetson Orin NX, running the entire navigation stack on-board.

Although the simulation used ROS 2 Jazzy (Ubuntu 24.04) and the physical robot used ROS 2 Foxy (Ubuntu 20.04), the same software architecture was maintained. This allowed parameters optimized under simulated camera noise to transfer directly to the physical platform without manual recalibration. RTAB-Map detected loop closures during walking, and the robot maintained a target velocity of 0.4 m/s. These results indicate that the proposed pipeline enables practical transfer of navigation parameters to the physical robot.

4.3 Limitations and Future Work

Several limitations remain. First, physical deployment bypasses the learned locomotion policy and uses the built-in Sport API; thus, this work validates the transfer of navigation and mapping parameters rather than the learned controller itself. Second, the experiments are conducted in a mostly static corridor, and obstacle avoidance is evaluated qualitatively without repeated-trial success rates. Future work will include quantitative benchmarks for localization drift, map consistency, obstacle-avoidance success rate, dynamic obstacles, and edge optimization.

5. CONCLUSION

This paper presented a 3DGS-based sim-to-real validation pipeline for quadrupedal visual navigation. By combining a photorealistic digital twin with RL-generated walking dynamics, the framework enables RTAB-Map and Nav2 parameters to be tuned under visual and motion disturbances before deployment. Experiments on a Unitree Go2 show that the optimized navigation stack can be deployed through a ROS 2 bridge and the built-in Sport API without manual recalibration.

ACKNOWLEDGEMENT

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2025-24683458).

USE OF AI TOOLS

During the preparation of this work, the authors used Gemini (developed by Google) for grammar editing and style improvement of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the final text of the paper.

REFERENCES

- [1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.
- [2] M. Labbé and F. Michaud, “RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation,” arXiv preprint arXiv:2403.06341, 2024.
- [3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [4] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Dretakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.
- [5] S. Zhu, L. Mou, D. Li, B. Ye, R. Huang, and H. Zhao, “VR-Robo: A real-to-sim-to-real framework for visual robot navigation and locomotion,” *IEEE Robotics and Automation Letters*, vol. 10, no. 8, pp. 7875–7882, August 2025.
- [6] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, “Sim-to-real: Learning agile locomotion for quadruped robots,” in *Proc. RSS*, 2018.
- [7] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Proc. CoRL*, 2022.
- [8] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras,” *IEEE Trans. Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [9] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *Proc. CVPR*, pp. 4104–4113, 2016.